

DATA STRUCTURES

Complexity Examples



1. Logarithmic loop: $O(\log n)$

```
for (int i = 1; i < n; i *= 2)
{
    cout << i;
}
```

Values of i

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow \dots$

Each iteration multiplies i by 2.

Where does $\log_2(n)$ come from?

After k iterations:

$$i = 2^k$$

The loop stops around n, so:

$$2^k \approx n$$

$$k = \log_2(n)$$

Example: n = 17

Printed values: 1, 2, 4, 8, 16

That is 5 iterations. Although $\log_2(17) \approx 4.09$, the exact loop count is the next integer. Big-O is still $O(\log n)$.

Therefore: $O(\log n)$

2. Decreasing by half is also $O(\log n)$

```
for (int i = n; i > 0; i /= 2) {  
    // work  
}
```

Example: $n = 32$

$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 0$

Every step divides by 2.

Derivation

After k divisions:

$$i = n / 2^k$$

At the end, i becomes about 1:

$$n / 2^k \approx 1$$

$$n \approx 2^k$$

$$k \approx \log_2(n)$$

Quick rule

If a loop changes like:

- $i = i \times 2$
- $i = i \times 3$
- $i = i / 2$
- $i = i / 10$

then the number of iterations is logarithmic: $O(\log n)$.

Therefore: $O(\log n)$

3. For loop: $j = j^2 \rightarrow O(\log \log n)$

```
for (int j = 2; j <= n; j = j * j) {  
    // work  
}
```

Watch the values carefully

$$\begin{aligned}j_1 &= 2 = 2^1 \\j_2 &= 2^2 = 4 \\j_3 &= 4^2 = 2^4 = 16 \\j_4 &= 16^2 = 2^8 = 256 \\j_5 &= 2^{16} = 65,536\end{aligned}$$

The exponent doubles each time!

Derivation

At the stopping point:

$$2^{(2^k)} \approx n$$

Take $\log_2$:

$$2^k \approx \log_2 n$$

Take $\log_2$ again:

$$k \approx \log_2(\log_2 n)$$

Therefore: $O(\log \log n)$

4. For loop: $j = j^3$ is still $O(\log \log n)$

```
for (int j = 2; j <= n; j = j * j * j)
{
    // work
}
```

Watch the values carefully

$$\begin{aligned}j_1 &= 2 = 2^1 \\j_2 &= 2^3 = 8 \\j_3 &= (2^3)^3 = 2^9 = 512 \\j_4 &= (2^9)^3 = 2^{27}\end{aligned}$$

Exponent sequence: $1 \rightarrow 3 \rightarrow 9 \rightarrow 27 \rightarrow \dots$

Derivation

At the stopping point:

$$2^{(3^k)} \approx n$$

Take $\log_2$:

$$3^k \approx \log_2 n$$

Take $\log_3$:

$$k \approx \log_3(\log_2 n)$$

Therefore: $O(\log \log n)$

5. Nested logarithmic loops: $O(\log n \cdot \log \log n)$

```
for (int i = n/2; i > 0; i /= 2) {  
    for (int j = 2; j <= n; j = j * j) {  
        // work  
    }  
}
```

Outer loop

$n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

Number of iterations $\approx \log_2 n$

Outer = $O(\log n)$

Inner loop

$j = 2 \rightarrow 4 \rightarrow 16 \rightarrow 256 \rightarrow \dots$

Number of iterations $\approx \log(\log n)$

Inner = $O(\log \log n)$

Total

The inner loop runs for every outer-loop iteration.

So, multiply the costs:

$O(\log n) \times O(\log \log n)$
 $= O(\log n \cdot \log \log n)$

6. Same nested idea with $j = j^3$

```
for (int i = 1; i <= n; i *= 2) {  
    for (int j = 2; j <= n; j = j * j * j)  
    {  
        // work  
    }  
}
```

Outer loop

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots \rightarrow n$

Outer iterations = $O(\log n)$

Inner loop

$2 \rightarrow 2^3 \rightarrow 2^9 \rightarrow 2^{27} \rightarrow \dots$

Inner iterations = $O(\log \log n)$
(the base 3 only changes constants)

Final answer

$O(\log n) \times O(\log \log n)$

$= O(\log n \cdot \log \log n)$

7. Nested loops where the inner loop runs i times

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= i; j++) {  
        // work  
    }  
}
```

Count the inner iterations

When $i = 1 \rightarrow 1$ time

When $i = 2 \rightarrow 2$ times

When $i = 3 \rightarrow 3$ times

...

When $i = n \rightarrow n$ times

Add them

Total work = $1 + 2 + 3 + \dots + n$

Sum of first n integers:

$$n(n + 1) / 2$$

$$= (n^2 + n) / 2$$

Why not $O(n)$?

Because the inner loop is NOT always n times.
It grows with i .

Therefore: $O(n^2)$

8. Three nested linear loops: $O(n^3)$

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) {  
        for (int k = 0; k < j; k++) {  
            // work  
        }  
    }  
}
```

Simple upper-bound method

Each loop is at most n iterations:

outer: $\leq n$

middle: $\leq n$

inner: $\leq n$

So total work is at most:

$$n \times n \times n = n^3$$

More precise view

Because $j \leq i$ and $k \leq j$, the exact count is smaller than n^3 (roughly a cubic sum).

But Big-O only needs an upper bound: $O(n^3)$

9. How to solve these questions in an exam

1

Look at the update

$i++ \rightarrow$ usually linear

$i *= 2 \rightarrow$ logarithmic

$i /= 2 \rightarrow$ logarithmic

$j = j*j \rightarrow \log \log$

2

Count each loop

Find how many times the loop body executes.

3

Nested? Multiply

If one loop is inside another, multiply their iteration counts.

4

Simplify

Drop constants and lower-order terms.

Golden rule

First understand how the variable changes. The update statement usually tells you the complexity.

10. The main thing to remember

If the value doubles / halves

→ $O(\log n)$

If the exponent itself grows fast

$j = j^2$ or $j = j^3$
→ $O(\log \log n)$

If loops are nested

Multiply their costs.

Example: $\log n \times \log \log n$

If loops add 1 each time

One loop → $O(n)$

Two nested → $O(n^2)$

Three nested → $O(n^3)$

Don't memorize the answer only

Write the sequence of values first.

Then derive the number of iterations.

That makes the Big-O answer much easier.